```python
import pandas as pd
import matplotlib.pyplot as plt
import scipy.signal as signal
import numpy as np
#import functions
```

```python
df = pd.read_csv('data.csv')
df.head()
```

	# time	# u	# y
0	0.000757976	73.05197899	0.093872626
1	0.010456612	59.35660805	1.213925581
2	0.020310119	77.84217147	2.302655267
3	0.030255519	82.03141879	3.413871346
4	0.040137494	83.18197025	4.478696609

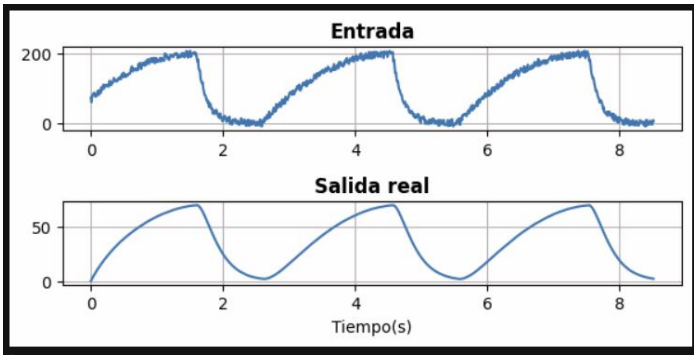
```python
time = df.time
u = df.u
y = df.y
```

```python
plt.figure(figsize = (6,3))

plt.subplot(2,1,1)
plt.plot(time, u)
plt.grid (True)
plt.title ('Entrada', fontweight = 'bold')

plt.subplot(2,1,2)
plt.plot(time, y)
plt.grid (True)
plt.title ('Salida real', fontweight = 'bold')
plt.xlabel ('Tiempo(s)')

plt.tight_layout()
plt.show()
```



```python
h_desc, _ = signal.deconvolve(y, u) #Siempre el primer parámetro debe ser la salida y el segundo la entrada SIEMPRE!!!
print(len(h_desc))
print(len(u))
```

```
1
863
```

```python
# ZERO PADDING: Agregar 0 al final
# y = [1, 2, 5, 8, 0, 9, 0, 0, 0, 0, 0, 0] -> ¿Cuántos 0 se agregan?: el valor de p, en este caso lo haremos con N/3
```

```python
round(len(u)*1/3)
```

```
288
```

```python
ypad = np.concatenate([y, np.zeros(round(len(u)*1/3))])
h, remainder = signal.deconvolve(ypad,u) #remainder = residuo (no lo usamos)

print(len(h)) # Es p + 1 --> El 1 que nos dió anteriormente + los 0 que agregamos

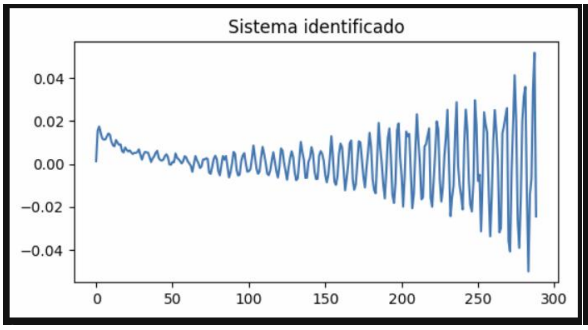
plt.figure(figsize =(6,3))
plt.plot(h) #TAREA: Graficar en función del tiempo --> Cuando no se pone variable independiente, el gráfica en función de las muestras
# Como el vector del tiempo es más largo que el sistema, este vector más grande se recorta al tamaño del más pequeño y se gráfica.
plt.title('Sistema identificado')

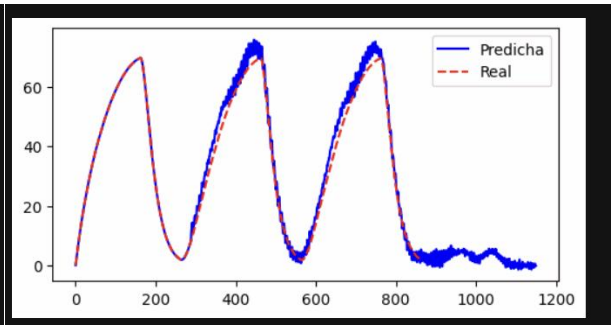
#¿Cómo se si si estoy haciendo bien las cosas?
# [si no tengo ecuaciones (no parametrica)]
# Convoluciono la entrada con el sistema hallado para obtener una salida que idealmente debe ser igual a la salida real.

# Revisión de identificación
y_pred_desc = signal.convolve (h, u)
plt.figure(figsize = (6,3))
plt.plot(y_pred_desc, label= 'Predicha', color = 'b')
plt.plot(y, label= 'Real', linestyle = '--', color = 'r')

plt.legend()
plt.show()

#Tarea: también gráficar vs el tiempo.
```





```python
import numpy as np


def LSE(t, u, y, p, N):
    """
    Least Squares Estimation (LSE) function.

    Parameters:
        t (numpy array): Time vector
        u (numpy array): Input signal
        y (numpy array): Output signal
        p (int): Number of parameters to estimate
        N (int): Number of data points to use

    Returns:
        h (numpy array): Estimated response
    """
    T = np.mean(np.diff(t))  # Mean time step
    UU = np.zeros((N, p))  # Matrix for regression

    for i in range(p):
        if i == 0:
            UU[:, 0] = u
        else:
            UU[i:, i] = u[: N - i]

    UU *= T  # Scale by sampling time
    AA = UU.T @ UU  # Compute normal equation matrix
    b = UU.T @ y  # Compute right-hand side
    h = np.linalg.solve(AA, b) * T  # Solve for h

    return h
```